

State awareness

Imagine your Windows installation went bonkers and started misbehaving. While reinstalling the OS is an option, it's the last one. You'll perform a number of checks such as running a virus scan and cleaning your registry; if all else fails, you're likely to use System Restore feature built into Windows. A real life-saver, System Restore will restore Windows to the state it was in at an earlier date and time. Yes, you can lose some programs and maybe some settings, but you won't have to take the pain of reinstalling the OS, and then reinstalling all the software on top of it.

What System Restore is basically doing is termed as 'state awareness' i.e. the process of capturing the state of the system including the installed programs and drivers, Windows settings and other crucial pieces of data. Later should you face a problem, you can choose one of your saved states that you know was working fine and system restore will automatically bring your Windows installation back to its saved state. Cool, eh? Not so much when you realise how much of a difficult feat this is to accomplish with embedded systems. Remember that installing anything on an embedded system is a lot more difficult than on a PC.

In case of an embedded system, component or system failures can cause huge losses, sometimes of lives. It's important for such systems to be able to go back to their original state (or a pre-saved state) if a transient failure occurs. Transient or impermanent failures can happen due to a software layer bug or voltage fluctuation. Such failures are impermanent and don't cause hardware damage but can temporarily disable the system.

In such cases it's important for the system to be able to recover itself to a previous workable state or its original state. This can be achieved by transferring the state to another system periodically or saving it on a storage unit locally. However for a state-aware design, it's important to save the state of components as well (along with the main system). As you might guess, it's the job of the software layer of the system to detect a corrupted state and be able to restore one from the past. While much of the state-aware design needs to be implemented at the software level, it's important that the hardware of which we need to save the state of has hardware features to support this.

Robustness

Robustness should not be confused with 'recoverability' which is the capability of recovering from a failure (and state-aware design does take care of this for the most part). Robustness is the attribute which helps prevent failure